

Лабораторная работа №8

Оптимизация

Доберштейн А. С.

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
3	Выводы	21

Список иллюстраций

2.1	Линейное программирование	5
2.2	Линейное программирование	5
2.3	Векторизованные ограничения и целевая функция оптимизации	6
2.4	Векторизованные ограничения и целевая функция оптимизации	6
2.5	Оптимизация рациона питания	7
2.6	Оптимизация рациона питания	7
2.7	Оптимизация рациона питания	8
2.8	Оптимизация рациона питания	8
2.9	Путешествие по миру	9
2.10	Путешествие по миру	9
2.11	Путешествие по миру	10
2.12	Портфельные инвестиции	10
2.13	Портфельные инвестиции	11
2.14	Портфельные инвестиции	11
2.15	Портфельные инвестиции	12
2.16	Портфельные инвестиции	13
2.17	Портфельные инвестиции	13
2.18	Восстановление изображения	14
2.19	Восстановление изображения	14
2.20	Восстановление изображения	15
2.21	Восстановление изображения	15
2.22	Восстановление изображения	15
2.23	Задание 1	16
2.24	Задание 2	17
2.25	Задание 2	17
2.26	Задание 3	18
2.27	Задание 3	18
2.28	Задание 4	19
2.29	Задание 4	19
2.30	Задание 5	20

1 Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

2 Выполнение лабораторной работы

Повторим примеры из лабораторной работы. (рис. 2.1 - 2.30).

```
[1]: # Подключение пакетов:
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK

@[32m@[1m Resolving@[22m@[39m package versions... •••

[2]: # Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

[2]: A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

[3]: # Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0)

[3]: y

[4]: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 100)
@constraint(model, 7x + 12y >= 120)

[4]:  $7x + 12y \geq 120$ 
```

Рис. 2.1: Линейное программирование

```
[5]: # Определение целевой функции:
@objective(model, Min, 12x + 20y)

[5]:  $12x + 20y$ 

[6]: # Вызов функции оптимизации:
optimize!(model)

[7]: # Определение причины завершения работы оптимизатора:
termination_status(model)

[7]: OPTIMAL::TerminationStatusCode = 1

[8]: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результата оптимизации:
@show objective_value(model);

value(x) = 14.999999999999993
value(y) = 1.25000000000000047
objective_value(model) = 205.0
```

Рис. 2.2: Линейное программирование

```

[9]: #Подключение пакетов:
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK

[10]: #Определение объекта модели с именем vector_model:
vector_model = Model{GLPK.Optimizer}

[10]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none

[13]: #Определение начальных данных:
A = [ 1 1 0 5;
      3 5 0 8;
      2 0 6 13]
b = [7; 3; 5]
c = [1; 3; 5; 2]

[13]: 4-element Vector{Int64}:
      1
      3
      5
      2

```

Рис. 2.3: Векторизованные ограничения и целевая функция оптимизации

```

[14]: #Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0)

[14]: 4-element Vector{VariableRef}:
      x[1]
      x[2]
      x[3]
      x[4]

[15]: #Определение ограничений модели:
@constraint(vector_model, A * x .== b)

[15]: 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64, 4}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
      x[1] + x[2] + 9 x[3] + 5 x[4] = 7
      3 x[1] + 5 x[2] + 8 x[4] = 3
      2 x[1] + 6 x[3] + 13 x[4] = 5

[16]: # Определение целевой функции:
@objective(vector_model, Min, c' * x)

[16]:  $x_1 + 3x_2 + 5x_3 + 2x_4$ 

[18]: #Вызов функции оптимизации:
optimize!(vector_model)

[19]: # Определение причины завершения работы оптимизатора:
termination_status(vector_model)

[19]: OPTIMAL::TerminationStatusCode = 1

[20]: # Демонстрация результата оптимизации:
@show objective_value(vector_model);

objective_value(vector_model) = 4.9230769230769225

```

Рис. 2.4: Векторизованные ограничения и целевая функция оптимизации

```
[21]: # Подключение пакетов:
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK

@[32m@[1m Resolving@[22m@[39m package versions... ***

[22]: # Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray{
  [1800 2200;
  91 Inf;
  0 65;
  0 1779],
  ["calories", "protein", "fat", "sodium"],
  ["min", "max"]}

# массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]

# Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
  [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59], foods}

[22]: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
      Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.5
 1.89
 2.09
 1.99
 2.49
 0.89
 1.59
```

Рис. 2.5: Оптимизация рациона питания

```
[23]: # Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
  [410 24 26 730;
  420 32 10 1190;
  560 20 32 1800;
  380 4 19 270;
  320 12 10 930;
  320 15 12 820;
  320 31 12 1230;
  100 8 2.5 125;
  330 8 10 180],
  foods,
  ["calories", "protein", "fat", "sodium"]}

[23]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
      Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
      Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
 410.0  24.0  26.0  730.0
 420.0  32.0  10.0  1190.0
 560.0  20.0  32.0  1800.0
 380.0   4.0  19.0   270.0
 320.0  12.0  10.0   930.0
 320.0  15.0  12.0   820.0
 320.0  31.0  12.0  1230.0
 100.0   8.0   2.5   125.0
 330.0   8.0  10.0   180.0

[24]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)

[24]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none
```

Рис. 2.6: Оптимизация рациона питания

```

[25]: # Определим массив:
categories = ["calories", "protein", "fat", "sodium"]

[25]: 4-element Vector{String}:
"calories"
"protein"
"fat"
"sodium"

[26]: # Определение переменных:
@variables(model, begin
    category_data[c, "min"] <= nutrition[c = categories] <=
    category_data[c, "max"]
    # Сколько покупать продуктов:
    buy[foods] >= 0
end)

[26]: (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
    Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
nutrition[calories]
nutrition[protein]
nutrition[fat]
nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
    Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{VariableRef}:
buy[hamburger]
buy[chicken]
buy[hot dog]
buy[fries]
buy[macaroni]
buy[pizza]
buy[salad]
buy[milk]
buy[ice cream])

```

Рис. 2.7: Оптимизация рациона питания

```

[27]: # Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))

[27]: 2.49buyhamburger + 2.89buychicken + 1.5buyhotdog + 1.89buyfries + 2.09buymacaroni + 1.99buypizza + 2.49buisalad + 0.89buymilk +
+

[28]: # Определение ограничений модели:
@constraint(model, [c in categories], sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

[28]: 1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape},1,...} with index sets:
    Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
-nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 330 buy[ice cream] = 0
-nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cream] = 0
-nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 10 buy[ice cream] = 0
-nutrition[sodium] + 730 buy[hamburger] + 1190 buy[chicken] + 1800 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 820 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 180 buy[ice cream] = 0

[29]: # Вывод функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)

[29]: OPTIMAL::TerminationStatusCode = 1

[30]: hcat(buy.data, JuMP.value.(buy.data))

[30]: 9x2 Matrix{AffExpr}:
buy[hamburger]  0.6045138888888888
buy[chicken]    0
buy[hot dog]    0
buy[fries]      0
buy[macaroni]   0
buy[pizza]      0
buy[salad]      0
buy[milk]       6.9701388888888935
buy[ice cream]  2.5913194444444441

```

Рис. 2.8: Оптимизация рациона питания

```
[31]: # Подключение пакетов:
import Pkg
Pkg.add("DelimitedFiles")
Pkg.add("CSV")
using DelimitedFiles
using CSV

@[32m@[1m Resolving@[22m@[39m package versions... •••

[37]: # Считывание данных:
passportdata = readlm(joinpath("passport-index-matrix.csv"),',,')

[37]: 200x200 Matrix{Any}:
"Passport"      "Albania"      -      "Afghanistan"
"Afghanistan"   "visa required" -1      "visa required"
"Albania"       -1             "visa required"
"Algeria"       "visa required" "visa required"
"Andorra"       90            "visa required"
"Angola"        "visa required" -      "visa required"
"Antigua and Barbuda" 90      "visa required"
"Argentina"     90            "visa required"
"Armenia"       90            "visa required"
"Australia"     90            "visa required"
"Austria"       90            -      "visa required"
"Azerbaijan"    90            "visa required"
"Bahamas"       90            "visa required"
⋮
"United Arab Emirates" 90      "visa required"
"United Kingdom"      90      "visa required"
"United States"       90      -      "visa required"
"Uruguay"             90      "visa required"
"Uzbekistan"          "visa required" "visa required"
"Vanuatu"             "visa required" "visa required"
"Vatican"             90      "visa required"
"Venezuela"           90      -      "visa required"
"Vietnam"             "visa required" "visa required"
"Yemen"              "visa required" "visa required"
"Zambia"             "visa required" "visa required"
"Zimbabwe"           "visa required" "visa required"
```

Рис. 2.9: Путешествие по миру

```
[38]: # Подключение пакетов:
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK

@[32m@[1m Resolving@[22m@[39m package versions... •••

[42]: # Задаём переменные:
cntr = passportdata[2:end,1]
vf = (x-> typeof(x)==Int64 || x == "VF" || x == "VOA" ? 1 :
0).(passportdata[2:end,2:end]);

[41]: # Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

[41]: A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

[43]: # Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass))

[43]: pass187 + pass188 + pass189 + pass190 + pass191 + pass192 + pass193 + pass194 + pass195 + pass196 + pass197 + pass198 + pass
```

Рис. 2.10: Путешествие по миру

```
[44]: # Вызов функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

[44]: OPTIMAL::TerminationStatusCode = 1

[45]: # Просмотр результата:
print(JuMP.objective_value(model), " passports:", join(cntr[findall(JuMP.value.(pass) .== 1)], ", "))

63.0 passports:Afghanistan, Andorra, Argentina, Australia, Azerbaijan, Bahrain, Brunei, Cambodia, Cameroon, Canada,
Chile, Colombia, Comoros, DR Congo, Djibouti, Equatorial Guinea, Eritrea, Fiji, Gabon, Georgia, Guinea, Guinea-Biss
au, Hong Kong, Hungary, Indonesia, Iraq, Ireland, Israel, Jamaica, Japan, Kuwait, Laos, Liberia, Libya, Macao, Mada
gascar, Malaysia, Maldives, Marshall Islands, Mauritania, Mauritius, Mongolia, Mozambique, Nauru, Nepal, New Zealan
d, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Solomon Islands, Somalia, South Sudan, Sri Lanka,
Syria, Taiwan, Timor-Leste, Togo, Turkmenistan, United States, Uruguay, Vietnam
```

Рис. 2.11: Путешествие по миру

```
[64]: # Подключение необходимых пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("XLXSX")
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Convex")
Pkg.add("SCS")
Pkg.add("Statistics")

@[32m@[1m Resolving@[22m@[39m package versions...•••••

[50]: # Считываем данные и размещаем их во фрейм:
T = DataFrame(XLSX.readtable("stock_prices.xlsx", "Sheet2"))

[50]: 13×3 DataFrame
      Row  MSFT  FB  AAPL
      Any  Any  Any
1  101.93  137.95  148.26
2  102.8   143.8   152.29
3  107.71  150.04  156.82
4  107.17  149.01  157.76
5  102.78  165.71  166.52
6  105.67  167.33  170.41
7  108.22  162.5   170.42
8  110.97  161.89  172.97
9  112.53  162.28  174.97
10 110.51  169.6   172.91
11 115.91  165.98  186.12
12 117.05  164.34  191.05
13 117.94  166.69  189.95
```

Рис. 2.12: Портфельные инвестиции

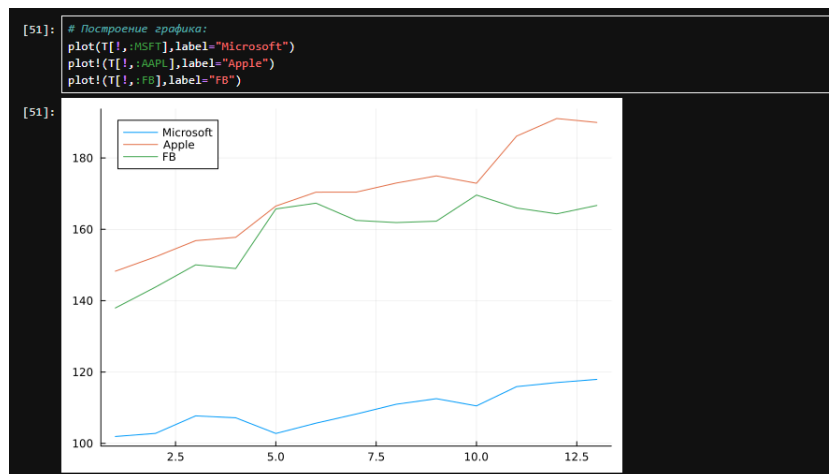


Рис. 2.13: Портфельные инвестиции

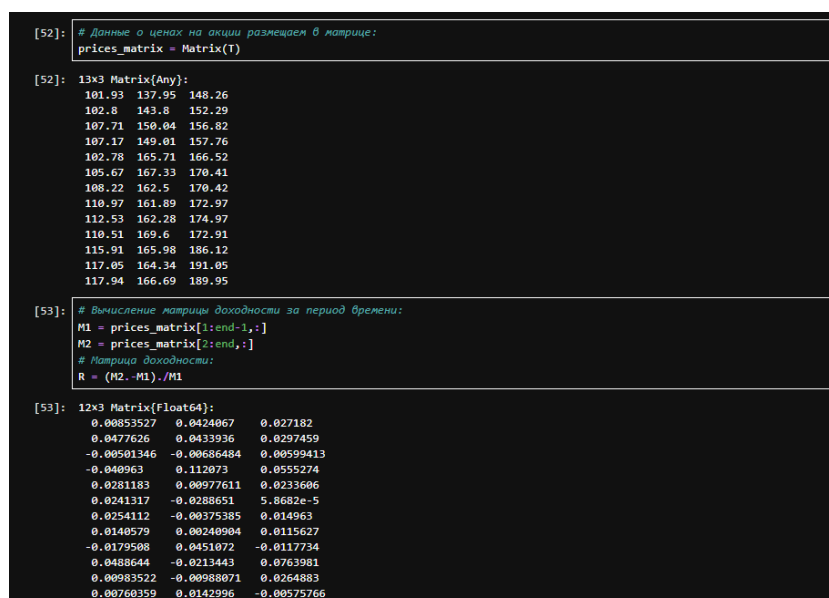


Рис. 2.14: Портфельные инвестиции

```

[93]: using DataFrames
      using XLSX
      using Plots
      using Convex
      using SCS
      using Statistics

[72]: # Матрица рисков:
      risk_matrix = cov(R)

[72]: 3x3 Matrix{Float64}:
      0.000659383  -0.000630653  0.000139112
      -0.000630653  0.00152162  0.000192288
      0.000139112  0.000192288  0.000635503

[73]: # Проверка положительной определенности матрицы рисков:
      isposdef(risk_matrix)

[73]: true

[74]: # Доход от каждой из компаний:
      r = mean(R, dims=1)[: ]

[74]: 3-element Vector{Float64}:
      0.012532748705136576
      0.016563036855293173
      0.021145804655032917

[75]: # Вектор инвестиций:
      x = Variable(length(r))

[75]: Variable
      size: (3, 1)
      sign: real
      vexity: affine
      id: 149_370

```

Рис. 2.15: Портфельные инвестиции

```
[76]: # Объект модели:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])

# Находим решение:
solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 11.44 seconds, 826.956 MiB of memory allocated
-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 12
cones:    z: primal zero / dual free vars: 1
          l: linear vars: 4
          q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 22, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
  0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 3.60e-03
 75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 1.24e-02
-----
status: solved
timings: total: 1.24e-02s = setup: 1.27e-04s + solve: 1.23e-02s
          lin-sys: 4.85e-05s, cones: 3.10e-05s, accel: 1.00e-05s
-----
objective = 0.000462
-----

[76]: Problem statistics
      problem is DCP           : true
      number of variables     : 1 (3 scalar elements)
      number of constraints    : 5 (5 scalar elements)
      number of coefficients   : 19
      number of atoms          : 14

Solution summary
  termination status : OPTIMAL
  primal status      : FEASIBLE_POINT
  dual status        : FEASIBLE_POINT
  objective value     : 0.0005

Expression graph
```

Рис. 2.16: Портфельные инвестиции

```
[78]: x

[78]: Variable
      size: (3, 1)
      sign: real
      vexity: affine
      id: 149_370
      value: [0.07740709795031134, 0.11606771003983696, 0.8065276266877964]

[79]: sum(x.value)

[79]: 1.0000024346779448

[80]: r'*x.value

[80]: 1x1 adjoint(::Vector{Float64}) with eltype Float64:
      0.0199472331085319

[81]: x.value .* 1000

[81]: 3x1 Matrix{Float64}:
      77.40709795031134
      116.06771003983695
      806.5276266877964
```

Рис. 2.17: Портфельные инвестиции

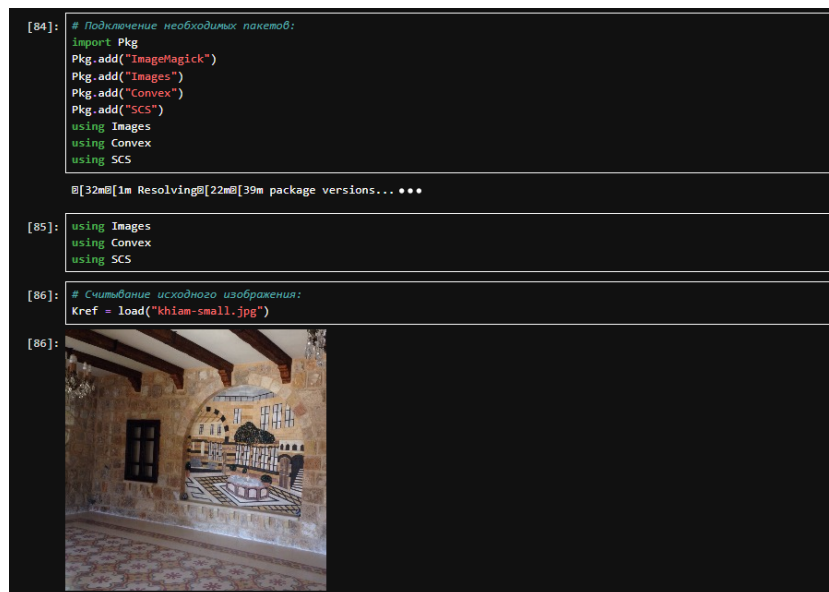


Рис. 2.18: Восстановление изображения

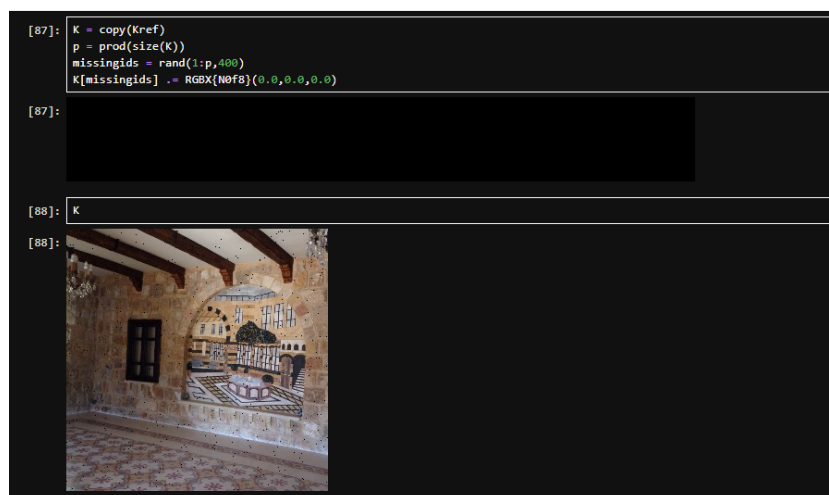


Рис. 2.19: Восстановление изображения

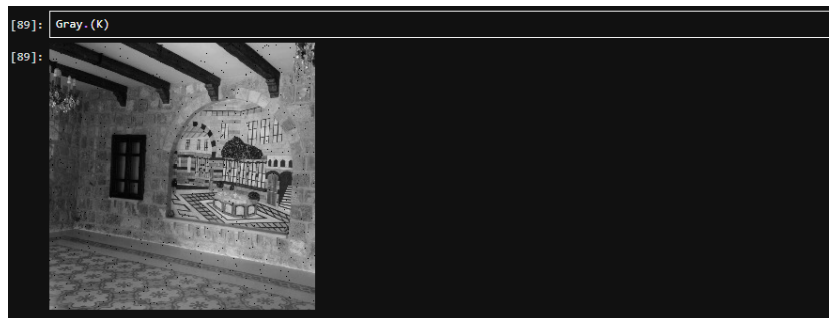


Рис. 2.20: Восстановление изображения

```
[90]: # Матрица ответов:
      Y = Float64.(Gray.(K));

[94]: correctids = findall(Y[:,].!=0)
      X = Convex.Variable(size(Y))
      problem = minimize(nuclearnorm(X))
      problem.constraints += X[correctids]==Y[correctids]

[94]: 1-element Vector{Constraint}:
      == constraint (affine)
      └─ + (affine; real)
          └─ index (affine; real)
              └─ 283x283 real variable (id: 381_639)
                  └─ 79691x1 Matrix{Float64}

[98]: # Находим решение:
      solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 5.65 seconds, 436.008 MiB of memory allocated

SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 80090, constraints m: 159781
cones:    z: primal zero / dual free vars: 79691
          nuc: nuclear vars: 80090, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 159781, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 1.27e+02 | 1.17e+01 | 3.09e+03 | 1.53e+03 | 1.00e-01 | 3.98e-01
150 | 4.10e-03 | 6.10e-05 | 1.11e-04 | 4.45e+02 | 9.29e-03 | 2.01e+01
-----
status: solved
timings: total: 2.01e+01s = setup: 2.54e-01s + solve: 1.99e+01s
```

Рис. 2.21: Восстановление изображения



Рис. 2.22: Восстановление изображения

```

[100]: # 1. Линейное программирование

model = Model(GLPK.Optimizer)

[100]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none

[101]: @variable(model, 0<=x1<=10)
      @variable(model, x2>=0)
      @variable(model, x3>=0)

      @constraint(model, -x1+x2+3x3 <= -5)
      @constraint(model, x1+3x2-7x3 <= 10)

      @objective(model, Max, x1+2x2+5x3)

[101]:  $x_1 + 2x_2 + 5x_3$ 

[102]: optimize!(model)

      termination_status(model)

[102]: OPTIMAL::TerminationStatusCode = 1

[103]: @show value(x1);
      @show value(x2);
      @show value(x3);

      @show objective_value(model);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375
objective_value(model) = 19.0625

```

Рис. 2.23: Задание 1

```

[104]: # 2. Линейное программирование. Использование массивов
vector_model = Model(GLPK.Optimizer)

[104]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none

[105]: A = [-1 1 3; 1 3 -7]
      b = [-5; 10]
      c = [1; 2; 5]

[105]: 3-element Vector{Int64}:
      1
      2
      5

[107]: @variables(vector_model, begin
      X[1:3]>=0
      end)

[107]: (VariableRef{X[1], X[2], X[3]},)

[108]: @constraint(vector_model, A * X .<= b)
      @constraint(vector_model, X[1] <= 10)

      @objective(vector_model, Min, c' * X)

[108]:  $X_1 + 2X_2 + 5X_3$ 

```

Рис. 2.24: Задание 2

```

[109]: optimize!(vector_model)

      termination_status(vector_model)

[109]: OPTIMAL::TerminationStatusCode = 1

[112]: @show value(x1);
      @show value(x2);
      @show value(x3);

      @show objective_value(vector_model);

      value(x1) = 10.0
      value(x2) = 2.1875
      value(x3) = 0.9375
      objective_value(vector_model) = 5.0

```

Рис. 2.25: Задание 2

```
[113]: # 3. Выпуклое программирование

n = 3
m = 4
A = rand(1:100, m, n);
b = rand(1:100, m);

[114]: x = Variable(n)
problem = minimize(sumsquares(A*x-b))
constraints = [x>=0]
solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 3.93 seconds, 229.697 MiB of memory allocated
-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 4, constraints m: 6
cones:    q: soc vars: 6, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 14, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 1.57e+03 | 1.00e+00 | 2.22e+03 | -1.10e+03 | 1.00e-01 | 1.80e-03
250 | 1.35e+01 | 9.34e-02 | 1.42e+01 | 2.81e+03 | 1.02e+00 | 3.12e-02
500 | 1.48e+02 | 5.99e-01 | 3.83e+01 | 6.26e+03 | 3.18e-01 | 6.07e-02
625 | 1.69e-03 | 1.92e-05 | 2.15e-02 | 3.94e+03 | 3.18e-01 | 7.54e-02
-----
status: solved
timings: total: 7.54e-02s = setup: 9.49e-05s + solve: 7.53e-02s
          lin-sys: 2.96e-04s, cones: 1.62e-04s, accel: 2.47e-04s
-----
objective = 3936.604400
-----
```

Рис. 2.26: Задание 3

```
[115]: problem.optval

[115]: 3936.5936487828303
```

Рис. 2.27: Задание 3

```

[124]: # 4. Оптимальная рассадка по залам

priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:1000]
priorities = hcat(priorities...)
priority_weights = sum(priorities, dims = 1)

[124]: 1x5 Matrix{Int64}:
 3731238 3961223 3921266 4441113 3951160

[119]: model1 = Model(GLPK.Optimizer);

[120]: @variable(model1, 180<=t1<=250)
@variable(model1, 180<=t2<=250)
@variable(model1, t3 == 220)
@variable(model1, 180<=t4<=250)
@variable(model1, 180<=t5<=250)

[120]: t5

[121]: @constraint(model1, t1+t2+t3+t4+t5==1000)

@objective(model1, Min, t1 * priority_weights[1] +
                t2 * priority_weights[2] +
                t3 * priority_weights[3] +
                t4 * priority_weights[4] +
                t5 * priority_weights[5])

[121]: 3811277t1 + 4121186t2 + 3841220t3 + 4081174t4 + 4151143t5

```

Рис. 2.28: Задание 4

```

[122]: optimize!(model1)

termination_status(model1)

[122]: OPTIMAL::TerminationStatusCode = 1

[123]: @show value(t1);
@show value(t2);
@show value(t3);
@show value(t4);
@show value(t5);

@show objective_value(model1);

value(t1) = 240.0
value(t2) = 180.0
value(t3) = 220.0
value(t4) = 180.0
value(t5) = 180.0
objective_value(model1) = 3.98340542e9

```

Рис. 2.29: Задание 4

```

[150]: # 5. План приготовления кофе

coffee = ["Raf", "Cappuccino"]
products = ["Grains", "Milk", "Sugar"]

costs = JuMP.Containers.DenseAxisArray([400, 300], coffee)

[150]: 1-dimensional DenseAxisArray{Int64,1,...} with index sets:
        Dimension 1, ["Raf", "Cappuccino"]
And data, a 2-element Vector{Int64}:
 400
 300

[151]: coffee_data = JuMP.Containers.DenseAxisArray(
        [40 140 5;
         30 120 0],
        coffee,
        products);

[152]: store = JuMP.Containers.DenseAxisArray([500, 2000, 40], products)

        coffee_data["Raf", "Grains"]
        store["Grains"]

[152]: 500

[156]: model_coffee = Model(GLPK.Optimizer);

        @variable(model_coffee, buy[coffee] >= 0)

        @objective(model_coffee, Max, sum(costs[c]*buy[c] for c in coffee))
        @constraint(model_coffee, [p in products], sum(coffee_data[c, p]*buy[c] for c in coffee) <= store[p])

        JuMP.optimize!(model_coffee)
        term_status = JuMP.termination_status(model_coffee)
        hcat(buy.data, JuMP.value.(buy.data))

[156]: 2x2 Matrix{AffExpr}:
 buy[Raf]      8
 buy[Cappuccino] 6

```

Рис. 2.30: Задание 5

3 Выводы

В результате выполнения работы были освоены пакеты Julia для решения задач оптимизации.